AAS | IOP Astronomy

Chapter 7

Radiative Transfer

> Nel mezzo del cammin di nostra vita, mi ritrovai per una selva oscura, ché la diritta via era smarrita.
>
> When halfway through our life's journey, I found myself in a gloomy wood, because the path which led straight had been lost.
>
> —Dante Alighieri

Our human perception of the Universe is dominated by electromagnetic radiation. Radiation forms the basis for essentially all astronomical research, including all of the great images reported by observational astronomers. Radiative transport is the foundation for understanding how radiation interacts with matter. Solving the differential equations describing the interaction between radiation and matter is no trivial task. In AMUSE, radiative transfer is addressed numerically using two quite distinct techniques: by photon propagation through a grid, and by Monte Carlo ray-tracing. These approaches are complementary. In AMUSE, these codes are typically used to determine the temperature and ionization state of the gas. Radiative transfer is expensive in terms of computer time, particularly if the timescales are governed by the hydrodynamics or the gravitational dynamics of the environment.

7.1 In a Nutshell

Radiative transfer deals with the way in which electromagnetic radiation travels through space and interacts with matter. Radiation is mediated by photons, which have energy, direction, and polarization state. Because the number of photons is generally very large, we normally work with statistical quantities, such as mean intensity and flux, that describe the bulk properties of the radiation field. These quantities may be frequency-dependent, or integrated over some portion of the electromagnetic spectrum. Radiation travels freely through vacuum without loss of energy, but when a ray interacts with matter, energy can be injected or removed.

doi:10.1088/2514-3433/adfc3cch7

Radiative flux. The radiative flux F is defined as the total amount of energy passing through a unit of area of some surface per unit of time. Flux is measured in units of erg s^{-1} cm^{-2}, or the SI equivalent (Wm^{-2}). If (as is usually the case) the energy is distributed over a range of frequencies, it is convenient to work in terms of F_ν (unit erg s^{-1} cm^{-2} Hz^{-1}), where $F_\nu d\nu$ is the energy flux in the frequency range $[\nu, \nu + d\nu)$. In that case, the total (bolometric) flux is

$$F = \int_0^\infty F_\nu \, d\nu. \tag{7.1}$$

Radiative intensity. In general, a radiation field will consist of photons moving in all directions at any point in space. The specific intensity I is the radiative flux per steradian,

$$I \equiv \frac{F}{\Delta\Omega}, \tag{7.2}$$

where $\Delta\Omega$ is solid angle around some direction $\hat{\boldsymbol{s}}$. Here, we adopt the convention that the unit vector $\hat{\boldsymbol{s}}$ represents a direction at any point in space in which radiation moves. Later, we will use the scalar s to describe distance along the direction (ray) $\hat{\boldsymbol{s}}$. For a frequency-dependent field, the radiative intensity is $I_\nu(\boldsymbol{x}, \hat{\boldsymbol{s}})$ (units erg s^{-1} cm^{-2} Hz^{-1} $ster^{-1}$).

Now, imagine an element of area with normal unit vector $\hat{\boldsymbol{n}}$. The radiative flux crossing this area moving outward (i.e., in the direction $\hat{\boldsymbol{n}}$) is

$$F_\nu = \int I_\nu \cos\theta \, d\Omega, \tag{7.3}$$

where $\cos\theta = \hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{s}} > 0$. For a locally isotropic field, $F_\nu = \pi I_\nu$. In vacuum, the intensity I_ν is constant. For radiation from a point source, the flux at distance r from a source scales as $1/r^2$. The constancy of the intensity along the path in vacuum plays a fundamental role in radiative transport and forms the basis for the radiative-transfer equation.

Blackbody radiation. The blackbody radiation is the radiation field emitted by a perfect emitter at temperature T. The intensity of radiation from that surface is

$$I_\nu = B_\nu(T) = \frac{2h\nu^3}{c^2}(e^{h\nu/kT} - 1)^{-1}, \tag{7.4}$$

independent of emission angle. Inverting this equation allows us to determine the temperature of a blackbody emitter by measuring the intensity I_ν in some direction. In practice, an emitting object is not a perfect blackbody, but has some thermal emissivity ε_ν. The thermal flux emitted from its surface [Equation 7.3] then is

$$F_\nu = \varepsilon_\nu \pi B_\nu(T). \tag{7.5}$$

7.1.1 Underlying Equations

In vacuum, the intensity I_ν is constant, but when radiation interacts with matter, I_ν changes. Matter along a ray may absorb radiation, decreasing I_ν, or emit energy,

increasing I_ν. In addition, radiation can be scattered by the medium—changed in direction but not in energy. The resulting *transfer equation* describes these processes. Many excellent books have been written on this subject (Chandrasekhar 1960; Peraiah 2001), often developing the details in specific geometries, such as the spherically symmetric and locally planar atmospheres of stars. Here, we will avoid specific geometries because we are mostly interested in the general problem, where no simplifying assumptions exist.

7.1.1.1 Absorption

Consider first a beam of radiation passing through a purely absorbing medium. As the beam traverses distance δs, the probability that a photon of frequency ν will be absorbed is $\delta P = \alpha_\nu \delta s$. The absorption coefficient α_ν can be expressed in terms of the density ρ and opacity κ_ν of the medium: $\alpha_\nu = \rho\kappa_\nu$. Thus, the fractional change in intensity of the beam is $\delta I_\nu / I_\nu = -\alpha_\nu \delta s$, so

$$\frac{dI_\nu}{ds} = -\alpha_\nu I_\nu. \tag{7.6}$$

The solution to this equation is

$$I_\nu(s) = I_\nu(0)\ e^{-\tau_\nu(s)}, \tag{7.7}$$

where $\tau_\nu(s) = \int_0^s \alpha_\nu(s')\ ds'$ is the *optical depth* of the medium.

If $\tau_\nu \ll 1$, absorption is negligible and the medium is said to be *optically thin*. If $\tau_\nu \gtrsim 1$, the medium is *optically thick*. In terms of $d\tau_\nu = \alpha_\nu ds$, Equation 7.6 becomes

$$\frac{dI_\nu}{d\tau_\nu} = -I_\nu. \tag{7.8}$$

The absorption coefficient α_ν has dimensions of inverse length. The distance a photon of frequency ν is expected to travel before being absorbed is the mean free path, $l_{\nu,\mathrm{free}} = 1/\alpha_\nu$.

7.1.1.2 Emission

In an optically thin medium, absorption may be ignored and the change in the intensity I along some trajectory (ray) depends only on the emission j_ν (unit erg s^{-1} cm^{-3} $\mathrm{Hz}^{-1}\,\mathrm{ster}^{-1}$) by material along that line. For isotropic emission, the intensity changes according to

$$\frac{dI_\nu}{ds} = j_\nu(s). \tag{7.9}$$

If j_ν is independent of I_ν, integrating Equation 7.9 gives

$$I_\nu(s) = I_\nu(0) + \int_0^s j_\nu(s')\ ds'. \tag{7.10}$$

The first term on the right-hand side is the incoming (or background) intensity, which in astronomical situations is often either absent or due to a background star or galaxy. The second term describes the total emission between 0 and s.

For a medium with both absorption and emission, Equations (7.6) and (7.9) combine to give

$$\frac{dI_\nu}{ds} = j_\nu(s) - \alpha_\nu(s) I_\nu, \tag{7.11}$$

which is easily solved:

$$I_\nu(s) = I_\nu(0) e^{-\tau_\nu(0,\, s)} + \int_0^s j_\nu(s') e^{-\tau_\nu(s',\, s)}\, ds', \tag{7.12}$$

where $\tau(s_0, s_1) = \int_{s_0}^{s_1} \alpha_\nu\, ds$ is the optical depth between s_0 and s_1. The first term on the right-hand side of Equation 7.12 is the attenuated input field; the second may be identified as the (partly reabsorbed) radiation emitted by the medium. Often, the radiative-transfer equation is rewritten in terms of the *source function* $S_\nu \equiv j_\nu / \alpha_\nu$:

$$\frac{dI_\nu}{ds} = \alpha_\nu(s)[S_\nu(s) - I_\nu]. \tag{7.13}$$

A system is in local thermodynamic equilibrium (LTE) if the local gas temperature equals the local Planck temperature of the radiation field (see Section 5.1.2.1). For a medium in LTE at temperature T, it can be shown that $j_\nu = \alpha_\nu B_\nu(T)$, so $S_\nu = B_\nu[T(s)]$. However, many of the systems we will study are not in LTE. Indeed, the whole idea of performing radiative-transfer simulations is to be able to make qualitative and quantitative statements about systems that may be far from LTE. This does not pose a serious limitation to our numerical solution, but it does make it difficult to validate our results from first principles.

7.1.1.3 Scattering

Pure absorption heats the absorbing medium, while pure emission cools it. A medium may also scatter radiation. Scattering may be thought of as absorption followed immediately by re-emission of the absorbed energy: it neither heats nor cools the gas, so it is unimportant for the local temperature structure of the medium. However, it is often critical in determining the details of the radiation field—and hence the appearance of a region.

The removal of photons from a beam due to scattering can be described by a scattering absorption coefficient $\alpha_\nu^{\rm scat} = \kappa_\nu^{\rm scat} \rho$, where $\kappa_\nu^{\rm scat}$ is the scattering opacity. Thus, the total absorption coefficient has contributions from both pure absorption and from scattering,

$$\alpha_\nu = \alpha_\nu^{\rm abs} + \alpha_\nu^{\rm scat}. \tag{7.14}$$

Similarly, the emission j_ν has contributions from emission and from scattering,

$$j_\nu = j_\nu^{\rm emis} + j_\nu^{\rm scat}. \tag{7.15}$$

Here, $j_\nu^{\rm emis}$ corresponds to the absorption coefficient $\alpha_\nu^{\rm abs}$, so $j_\nu^{\rm emis} = \alpha_\nu^{\rm abs} B_\nu(T)$ in LTE.

The expression for $j_\nu^{\rm scat}$ introduces considerable complexity into the transfer equation because it couples beams moving in all directions at any point. We denote by $p_\nu(\hat{\boldsymbol{s}}, \hat{\boldsymbol{s}}')$ the probability that a photon initially moving in direction $\hat{\boldsymbol{s}}'$ is scattered into direction $\hat{\boldsymbol{s}}$ (i.e., into the direction of the beam under consideration in the transfer equation), where $\int p_\nu(\hat{\boldsymbol{s}}, \hat{\boldsymbol{s}}')\, d\Omega' = 1$. It follows that

$$j_\nu^{\rm scat} = \int \alpha_\nu^{\rm scat} I_\nu(\hat{\boldsymbol{s}}')\, p_\nu(\hat{\boldsymbol{s}}, \hat{\boldsymbol{s}}')\, d\Omega'. \tag{7.16}$$

Scattering turns a relatively simple ordinary differential equation into an integro-differential equation that is much harder to solve.

The physics of the scattering interaction is contained entirely within the function p_ν. If the medium is composed of particles with radii a much smaller than the wavelength ($\lambda = c/\nu$) of the radiation, the interaction is in the so-called Rayleigh regime, which is well understood from a theoretical perspective (Bohren & Huffman 1983). If the particles have radii much larger than the wavelength ($a \gg \lambda$), as is generally the case with dust particles, then the scattering cross section is $\sigma \sim \pi a^2$ and we are in the geometric optics regime. Radiative transfer through a medium of large particles can be complex due to effects at the edges of the particles, or if the particles are partially transparent or (usually) not spherical. Radiative transfer through a medium composed of particles with sizes comparable to the wavelengths of interest ($a \simeq \lambda$) is most challenging, and we refer the interested reader to Chandrasekhar's textbook on the topic (Chandrasekhar 1960).

7.1.2 Physics of the Integrator

When numerically solving the radiative-transfer equation [Equation 7.11], the objective may be to construct an image of the simulation, compute the energy spectrum, or determine the temperature and ionization structure of a gaseous body. These are rather different objectives, and a wide range of numerical methods have been developed to address each.

Radiative transfer is difficult because the emission j_ν and the absorption α_ν are generally not known in advance—the radiation field $I_\nu(\mathbf{x}, \hat{\boldsymbol{s}})$ we wish to compute affects both. Local absorption and emission of energy can change the temperature and ionization structure of the medium, in turn producing large changes in j_ν and/or α_ν. At the same time, all the rays from any direction that pass through location $\mathbf{x}$ contribute via scattering to the transfer equation at $\mathbf{x}$. Even if scattering can be neglected, in a system with multiple sources (e.g., a star-forming region), conditions at any location depend on the solution to the transfer equation along multiple directions. In other words, radiation from one region of a molecular cloud affects conditions in another part of the cloud, which in turn may affect conditions in the original region. Solving this problem is a major challenge in computational radiative transfer.

Photons travel at the speed of light. For the problems at hand, we assume that the solution to the radiative-transfer equation is established instantaneously each time the solver is called. The underlying assumption is that the effective travel time of a photon is much shorter than the timescale for local changes in the scattering medium

though which the photons travel. We do not expect that this assumption will pose a problem or affect simulation results in any negative way (meaning that the consequences of the assumption are likely smaller than the uncertainties in the numerical scheme).

However, we know that the photon travel time can be long, in some cases, compared to the material response time. In such cases, delay times may be important. For example, it may be the case that the matter needs some characteristic time to respond by cooling down or heating up. In material with large optical depth, radiation may undergo many absorptions and re-emissions and the accumulation of short response times can become important. Although probably not very relevant in astrophysics, this aspect of radiative transfer has been included in the study of photonic crystals (Baba 2008).

7.2 Radiative Transfer in AMUSE

7.2.1 Radiative-transfer Modules

Table 7.1 lists the three radiative-transfer codes currently available as standard in AMUSE. Note that it does not include the radiative-transfer routines built into the `Flash` (Fryxell et al. 2000) hydro module; see Chapter 6. The codes listed differ in their description of both the radiation field and the medium with which it interacts, but all employ essentially the same iterative process to solve the transfer equation and determine the intensity field $I_\nu(\mathbf{x})$ at each spatial location.

A starting approximation to $I_\nu(\mathbf{x})$ is typically the solution obtained during the previous invocation of the module. Alternatively, we might simply set $I_\nu = 0$ everywhere. The emission $j_\nu(\mathbf{x})$ is then calculated at every location, taking into account scattering from the existing field, emission from the gas, and injection of energy from stars and other external sources. Finally, the transfer equation [Equation 7.11] is solved along each photon trajectory to give a new estimate for $I_\nu(\mathbf{x})$. The matter density is held static while the radiation field is updated, but its temperature and ionization structure (and hence its emission and absorption) change at each iteration in response to the changing radiative environment. As illustrated in Figure 7.1, this iterative process stops when changes to I_ν become acceptably small.

We now discuss some specifics of the AMUSE modules listed in Table 7.1. Although the AMUSE philosophy is to hide such details whenever possible, the radiative-transfer modules are more complex than gravitational (see Chapter 4) or even hydrodynamical (see Chapter 6) codes, and a little inside knowledge can be useful in deciding which radiative module to use for a given problem.

Data structure. The three modules (see Table 7.1) differ widely in their description of the medium through which the radiation propagates. `Mocassin` bins the gas onto a Cartesian grid, although there is a compile option to use a Voronoi (1908) tessellated grid (Hubber et al. 2016), while `SimpleX` uses an unstructured Delaunay-tessellated grid (Delaunay 1934), with more grid points in regions of higher opacity. The tessellated versions works best with SPH hydro simulations whereas regular Cartesian grid methods would be more suitable in combination with grid-based hydrodynamics codes. `SPHray` has no grid, but rather (as the name

Table 7.1. Radiative-transfer Codes Incorporated into AMUSE

Module	Mocassin (ref. 1)	SimpleX (2)	SPHRay (3)
Language	Fortran 95	C++	Fortran 95
Parallelization	MPI	OPENMP	OPENMP
Data Structure	Cartesian grid	Unstructured Delaunay grid	No grid; SPH particle distribution
Radiation Representation	Monte Carlo energy packets	Photon numbers and directions	Monte Carlo energy packets
Ray Propagation	Absorption computed from transfer equation; (re)emission creates new packets	Photons transported to adjacent cells; absorption computed from transfer equation; (re)emission/ scattering adds/redirects photons	Absorption computed from transfer equation; (re)emission creates new packets
Time Dependence	Thermal and ionization equilibrium	Thermal and ionization equilibrium	Time-dependent temperature equation; nonequilibrium solution
Physics	Photoionization, photoelectric heating, and recombination; dust absorption	Photoionization, photoelectric heating, and recombination; dust absorption	Photoionization, photoelectric heating, and recombination

References: (1) Ercolano et al. (2003), (2) Altay et al. (2008); Paardekooper et al. (2010), (3) Ritzerveld & Icke (2006); Altay et al. (2008, 2011). In Section A.8, we present a more general overview of the various code application domains.

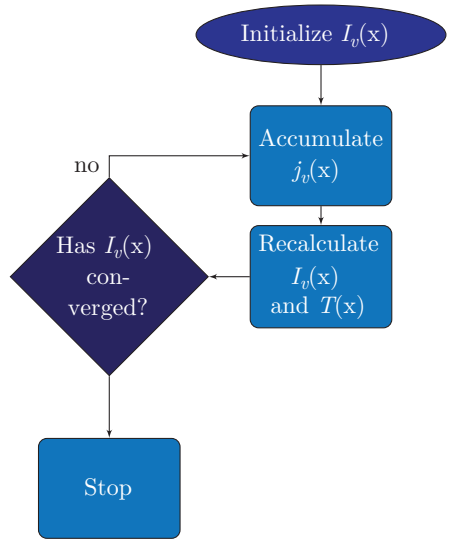

Figure 7.1. Workflow for a radiative-transfer code.

suggests) propagates photons through the interpolated density and temperature (as well as emission and absorption) fields defined by an SPH hydro simulation.

Radiation representation. `Mocassin` and `SPHray` are Monte Carlo methods, using a large number (typically millions) of energy packets to model the radiation field. The packets are created at sources and by emission/scattering events, and are removed by absorption as each packet propagates through the medium. `SimpleX` keeps track of the numbers of photons in each grid cell moving in the directions of the adjacent cell centers (a variable number, but averaging ~16 in three dimensions). Photons created by sources may be absorbed, emitted, or scattered as they move from cell to cell. Because only a limited number of directions is possible, the speed of the code is independent of the number of sources.

Ray propagation. In the Monte Carlo codes `Mocassin` and `SPHray`, each energy packet performs a random walk through the medium. Along each portion of a packet's trajectory, the absorption/scattering probability is

$$p(\tau) = e^{-\tau}, \tag{7.17}$$

where τ is optical depth measured along the path (Section 7.1.1.2). Operationally, the optical depth to the next absorption (or scattering) is determined by choosing a random number $\xi \in [0, 1)$ and setting $\tau = -ln(1 - \xi)$. The directions of emitted or scattered packets are chosen randomly (uniformly for emission or according to $p_\nu(\hat{\boldsymbol{s}}, \hat{\boldsymbol{s}}')$ for scattering; see Section 7.1.1.3). The process is repeated until the packet is absorbed or leaves the computational domain. `SimpleX` uses a similar optical depth formulation to determine the amount of absorption/scattering between one cell center and the next, it then modifies the photon distribution at each cell center to model emission and scattering. Because photons cannot move in straight lines on the irregular grid, care is taken to ensure that photons moving through optically thin regions on average maintain the correct direction.

Time dependence. `Mocassin` and `SimpleX` determine the physical conditions in the medium by assuming local radiative and ionization equilibrium at each iteration, and therefore return the steady-state solution to the problem after each invocation. `SPHray` solves an energy-balance equation for the temperature and ionization state of the medium as it computes the radiation field, and thus it generally returns an instantaneous nonequilibrium (time-dependent) solution.

Physics of the medium. All modules contain treatments of photoionization, photoelectric heating, recombination, and other directly related atomic/molecular cooling processes. Dust absorption is treated in all three radiative-transfer codes, although the implementations differ and not all internal possibilities are exposed to the current AMUSE interface. For `SPHray`, it is possible to set a stellar radiation spectrum, which is realized by means of a table that can be identified via the function

```
rt_code = Sphray()
rt_code.function get_spectra_file("spectra_file_name")
```

Normally, this file refers to a standard file in the AMUSE repository. It generally uses seven frequency channels to resolve a radiative source. Absorption in `SPHray` is computed from the transfer equation in each cell, after which re-emission creates new source. The transfer equation is solved using a semi-implicit iteration scheme (Iliev et al. 2006). `Mocassin` allows multiple sources with different spectra and can be used in conjunction with dust absorption, but this functionality is currently not supported by the `AMUSE` interface.

7.2.2 Ionization of a Molecular Cloud

We can run a radiative-transfer code in AMUSE in much the same way as running an N-body, stellar evolution, or hydrodynamics code. The procedure is a little more elaborate because radiative transfer requires two ingredients—a gaseous region through which the radiation transfer is to be calculated, and a list of ionizing sources. The typical application for radiative transfer in AMUSE will be star-forming regions, circumstellar disks, dense stellar winds, or supernova nebulae. The modules used to solve the radiative-transfer equations differ somewhat, and the user may have to adjust some of the fundamental code parameters before initiating the particular code.

Although the radiative-transfer codes in the AMUSE framework share essentially the same basic interface, they differ slightly in detail (see Table 7.1). Each is well suited to the problem of an embedded star cluster with radiative feedback. Code example 7.1 presents a simple example of how to start and run a radiative-transfer problem in AMUSE. It is not very elaborate, but it demonstrates the basic functionality and calling sequence for running a radiative-transfer problem. In this example, we opt for the `SPHRay` code, but we could have used either `Simplex` or `Moccasin` as well.

We start by creating a gaseous region—in this case, a homogeneous spherical distribution of $N = 10^4$ neutral gas particles slightly offset from the stellar source. This region is surrounded by a somewhat lower-mass disk with an outer radius of 5 pc (Code example 7.2). A specific internal energy of $(9\ \mathrm{km\ s^{-1}})^2$, but no ionizing radiative flux. We explicitly set the ionizing luminosity of the source particle to

```
def rad_minimal(
    number_of_particles=10000,
    luminosity=100 | units.LSun,
    boxsize=100 | units.parsec,
    time_end=0.1 | units.Myr,
    seed=12345,
):
    np.random.seed(seed)
    ism, converter = generate_ism_initial_conditions(number_of_particles, boxsize)

    source = Particle()
    source.position = (1, 0, 0) | units.parsec
    source.luminosity = luminosity / (20.0 | units.eV)

    radiative = Sphray(convert=converter, mode="openmp")
    radiative.set_boundary(0)  # vacuum
    radiative.set_spectra_file("spectra/thermal6e4.cdf")
    radiative.set_raynumber(1.0e4 | units.Myr**-1)
    print(radiative.parameters)

    radiative.src_particles.add_particle(source)
    radiative.gas_particles.add_particles(ism)

    radiative.evolve_model(time_end)

    print(radiative.gas_particles)
    mux = mu()
    temperature = mux / constants.kB * radiative.gas_particles.u

    print(f"min ionization: {radiative.gas_particles.xion.min()}")
    print(f"average ionization: {radiative.gas_particles.xion.mean()}")
    print(f"max ionization: {radiative.gas_particles.xion.max()}")
    plot_ionization_fraction(
        radiative.gas_particles.position, radiative.gas_particles.xion, temperature
    )

    plot_gas_temperature(radiative)

    radiative.stop()
```

Code example 7.1: Minimal routine to determine the global degree of ionization of a Plummer (1911) distribution of hydrogen gas with a scale radius of 0.5 pc and a mean density of 83 atoms cm^3. The initial conditions are generated using the code in Code example 7.2. Each of the particles was irradiated by a star with luminosity 100 $L_\odot$ for a time interval of 10^6 yr. This snippet was taken from the source code in `amuse.examples.rad_minimal`.

100 $L_\odot$, load the specific energy distribution for the star's spectral type, and indicate to what resolution, in terms of photons, we wish to integrate. The gas in the radiative-transfer code requires an ionization fraction `xion`, which we set to 0.5 (half the hydrogen atoms are already ionized).

The cloud is generated using

```
ism = new_plummer_gas_model(number_of_particles, converter)
ism.xion = 0.5
```

and the disk with

```
def generate_ism_initial_conditions(number_of_particles, boxsize):
    converter = nbody_system.nbody_to_si(1.0 | units.MSun, 0.5 | units.parsec)
    ism = new_plummer_gas_model(number_of_particles, converter)

    mass_disk = 0.2 * ism.mass.sum()
    n_disk = int(mass_disk / ism[0].mass)
    converter = nbody_system.nbody_to_si(mass_disk, 1 | units.parsec)

    from amuse.ext.protodisk import ProtoPlanetaryDisk

    disk = ProtoPlanetaryDisk(
        n_disk,
        convert_nbody=converter,
        densitypower=1.5,
        Rmin=1,
        Rmax=5,
        q_out=2.0,
        discfraction=0.1,
    ).result
    disk.x += 1 | units.pc
    ism.add_particles(disk)
    ism.xion = 0.5

    hydro = Fi(converter)
    hydro.gas_particles.add_particles(ism)
    hydro.evolve_model(1 | units.hour)
    hydro.gas_particles.new_channel_to(ism).copy()
    hydro.stop()
    ism = ism.select(lambda r: r.length() < 0.5 * boxsize, ["position"])
    print(
        f"Max density: {ism.rho.max().in_(units.MSun/units.parsec**3)} ",
        f"Mean density: {np.mean(ism.rho.value_in(units.g/units.cm**3))} g/cc ",
        f"{ism.rho.max().in_(units.amu/units.cm**3)}",
        f"{np.mean(ism.rho.value_in(units.amu/units.cm**3))} amu/cc",
    )
    return ism, converter
```

Code example 7.2: Routing for generating a gas distribution as initial conditions for the radiative-transfer calculation (see Code example 7.1). This snippet was taken from `amuse.examples.rad_minimal`.

```
from amuse.ext.protodisk import ProtoPlanetaryDisk
disk = ProtoPlanetaryDisk(
    number_of_particles,
    convert_nbody=converter,
    densitypower=1.5,
    Rmin=1,
    Rmax=10,
    q_out=1.0,
    discfraction=0.1,
).result
disk.h_smooth = 0.01 | units.pc
disk.x += 1 | units.pc
disk.xion = 0.5
```

which we center around the off-center source star at $x = 1\,\mathrm{pc}$.

In order to allow the radiative-transfer code to use the gas as a scattering medium, the local density and smoothing length of the gas particles must be determined. In principle, this parameter could have been determined by the Plummer- and disk-generating functions, but here we use an SPH code (Hernquist & Katz 1989; Pelupessy et al. 2004; Gerritsen & Icke 1997) to calculate the density for each SPH particle. In the following snippet, we instantiate the SPH code `Fi` and run it for a short while (1 minute in this example) before copying the relevant information back to the local interstellar medium particles,

```
hydro = Fi(converter)
hydro.gas_particles.add_particles(ism)
hydro.evolve_model(1 | units.minute)
hydro.gas_particles.new_channel_to(ism).copy()
```

For `SimpleX`, it is important to ensure that there is no scattering nor any sources outside the box, and that there is not too much vacuum inside the box. It can be tricky to choose the right box size. In a coupled simulation where the sources and the gas particles are moving, this can make it difficult to keep the numerical solver stable. In the ray-tracing radiative-transfer codes, `SPHray` and `Mocassin`, this is less of an issue. In our small illustration, we need only remove the particles that are outside the box, which we do via an array operation

```
ism = ism[ism.position.lengths() < 0.5 * boxsize]
```

We now can initialize the star as a radiative source, which is a single particle at the origin of our coordinate system. The radiative-transfer code requires the ionizing luminosity of the radiative source to be expressed in photons per second. We make the conversion, assuming that each photon deposits 20 eV, so if the total ionizing luminosity is `Lstar` (W), then the quantity passed to the radiative-transfer code is `Lstar/(20. | units.eV)`, or 1.2×10^{46} photon/s in this case,

```
source = Particle()
source.position = (1, 0, 0) | units.parsec
source.luminosity = Lstar / (20. | units.eV)
```

The source, typically, is not a scattering object, in which case we do not give it a degree of ionization, nor a density or smoothing length.

We could add the source to the gas particles as well as to the source particles, in which case it would also behave as a scattering and absorbing object. This would require us to specify values for the particle's mass (`mass`), luminosity, density (`rho`), ionization state (`xion`), smoothing length `h_smooth`, and internal energy (`u`), in much the same way as we did for the gas. In this example, we do not allow the source particle to scatter or absorb its own radiation.

At this point, we start the radiative-transfer code. For most radiative-transfer codes, we simply specify how many rays are desired. This sets the resolution of the solution. For our example, we decided to use 10^5 rays per Myr, which solves the problem in about 1 minute. Be aware, though, that the calculation timescales quite steeply with the number of rays, and with the number of SPH particles.

For SimpleX, which is a Delaunay-tessellated radiative-transfer solver, we have to specify the size of the box within which we want it to solve radiative-transfer equations. This will pose some interesting challenges later, when we combine the radiative solver with gravitational dynamics and/or hydrodynamics.

```
rad = Simplex()
rad.parameters.box_size = boxsize
```

As noted earlier, the correct operation of the SimpleX code is quite sensitive to the size of the box, and may require some experimentation

```
radiative.particles.add_particle(source)
radiative.particles.add_particles(ism)
```

It is interesting to note that, in contrast to the hydrodynamical solvers discussed in Section 6.2.2, some radiative-transfer codes have only one set of particles, whether they are sources or are part of the scattering medium (but see Section 8.6.1 for a counterexample).

We are now ready to solve the radiative-transfer equation by evolving the model.

```
rad.evolve_model(time_end)
```

Here, we integrate the radiative transfer to time_end = 1 Myr. This calculation is fast because it is carried out for a relatively small number of scattering and absorbing particles and a single radiative source. However, we will see that, when multiple sources are introduced and/or the scattering medium is resolved at higher resolution, this operation can quickly become far more computationally intensive than the gravitational dynamics, stellar evolution, and even the hydrodynamics calculations combined. For some problems, the radiation integration time interval may be as short as a few years, which is usually small compared to the timescales on which other processes operate (see Figure 1.5), making radiative transfer one of the most expensive operations in a multiphysics environment.

In Figures 7.2 and 7.3, we present the result of the simulation at an age of 1 Myr using SPHRay. By that time, the disk on the star's right side is fully ionized, but the cloud (the star's left) blocks the radiation, leaving the left side of the disk neutral. The ionization front, presented in Figure 7.2 is much broader than expected based on the theory, but the morphology of the adopted cloud is rather complex, compared to a homogeneous sphere commonly used in such theoretical models; this example does not lead to a clear Strömgren (1939) sphere. Another reason for the lack of a Strömgren sphere originates from the two assumptions made in our model: the finite resolution and the limited number of rays in the ray-tracing.

7.3 Examples

7.3.1 Ionization Front in an H_2 Region

Most radiative-transfer codes require a large number of inputs, usually including an ionizing source and an absorbing medium. The former is often a star, and the latter a

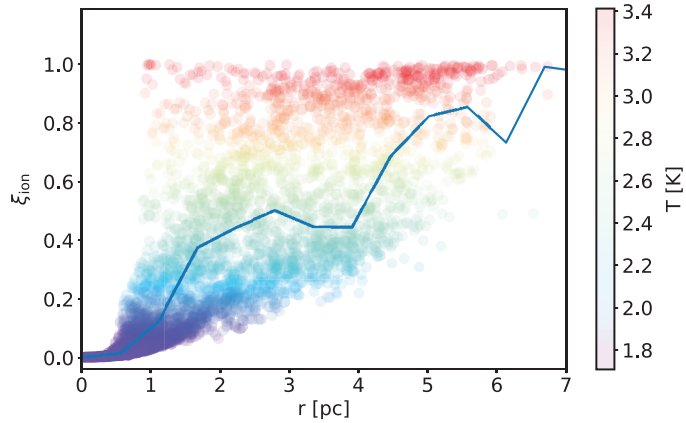

Figure 7.2. Degree of ionization ξ_{ion} due to a $100\,L_\odot$ star at the center of a 3 pc spherically symmetric Plummer distribution of 10^4 gas particles. The curve gives the average degree of ionization, binned in intervals of 0.1 pc. The figure was made using the script in Code example 7.1 and took 20 seconds on a laptop, see also `amuse.examples.rad_minimal`.

gaseous region. We addressed stellar evolution quite extensively in Chapter 5 and hydrodynamics of gaseous bodies in Chapter 6. If you skipped these chapters, this might be a good moment to revisit them before continuing.

In this example, we use stellar evolution and hydrodynamics together with radiative transfer to calculate the radial electron temperature structure of an initially homogeneous H_2 region with density 100 cm^{-3}, which is illuminated by a $120\,M_\odot$ (initial mass) star at an age of 3.3 Myr. At that age, such a star has, according to the `SeBa` stellar evolution code (Portegies Zwart & Verbunt 1996, 2012), a mass of 56.8 $M_\odot$, a luminosity of about $1.5 \times 10^6\,L_\odot$, and a temperature of 50,000 K. The stellar evolution was carried out using the simple script given in Code example 5.3. After the star is located at the cloud center, we initialize the gas distribution in a grid as in Code example 7.3. We then set up the radiative-transfer module; Code example 7.4 presents a snippet of code to do this using `Mocassin`. Of course, these initial conditions are a bit contrived. One might wonder how a 3.3 Myr old star of $120\,M_\odot$ could arrive in the middle of a cloud of molecular hydrogen, but anything is possible in the computer.

The number of settable parameters in a radiative-transfer code is quite large. Most of the fundamental code parameters used in radiative transfer can be queried from the helper function of the running code (see Section 2.2.1). In this example, we rely heavily on the code defaults and only set a few parameters explicitly to illustrate how this is done. Two important attributes are the grid resolution and the chemical abundances of the gas in the grid. In our example, we set these as follows:

```
setup_grid(outer_radius, Ngrid)
setup_abundancies(radiative_transfer)
```

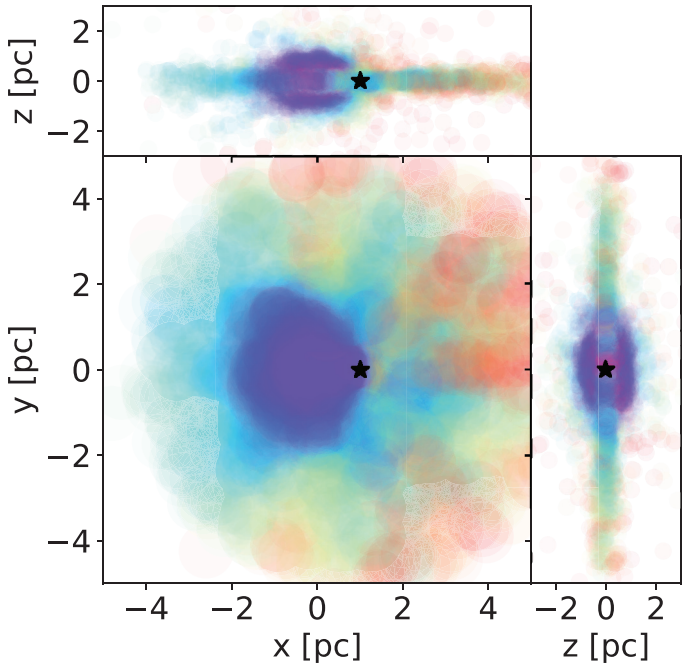

Figure 7.3. Top and side views of the gas distribution around the 100 $L_\odot$ star from the calculation presented in Figure 7.2. The bump to the left of the star shields the disk from radiation, and remains cold. The temperature distribution in the disk is radially structured, depending on clumpiness in the particle distribution in the disk. The color is coded according to the colorbar in Figure 7.2. `amuse.examples.rad_minimal`.

```
def make_grid(
    number_of_grid_cells, length, constant_hydrogen_density, inner_radius,
    outer_radius
):
    grid = new_regular_grid([number_of_grid_cells] * 3, length.as_vector_with_length
    (3))
    grid.radius = grid.position.lengths()
    grid.hydrogen_density = constant_hydrogen_density
    grid.hydrogen_density[grid.radius <= inner_radius] = 0 | units.cm**-3
    grid.hydrogen_density[grid.radius >= outer_radius] = 0 | units.cm**-3
    return grid
```

Code example 7.3: Initialization of a grid with a uniform-density spherical cloud.

Abundances of various elements are initialized as fractions relative to hydrogen. The following small snippet sets a few abundances, but printing the entire table will allow you to quickly develop an appreciation for the complexity of the chemical network available in these radiative-transfer codes.

```
def initiate_radiative_transfer_code(outer_radius, Ngrid):
    radiative_transfer = Mocassin(number_of_workers=4)

    radiative_transfer.set_input_directory(
        radiative_transfer.get_default_input_directory()
    )
    radiative_transfer.set_mocassin_output_directory(
        radiative_transfer.output_directory + os.sep
    )
    radiative_transfer.initialize_code()
    radiative_transfer.set_symmetricXYZ(True)

    setup_grid(radiative_transfer, outer_radius, Ngrid)
    setup_abundancies(radiative_transfer)

    radiative_transfer.parameters.initial_nebular_temperature = 6000.0 | units.K
    radiative_transfer.parameters.high_limit_of_the_frequency_mesh = (
        15 | mocassin_rydberg_unit
    )
    radiative_transfer.parameters.low_limit_of_the_frequency_mesh = (
        1.001e-5 | mocassin_rydberg_unit
    )

    radiative_transfer.parameters.total_number_of_photons = 10000000
    radiative_transfer.parameters.total_number_of_points_in_frequency_mesh = 600
    radiative_transfer.parameters.convergence_limit = 0.09
    radiative_transfer.parameters.number_of_ionisation_stages = 6
    radiative_transfer.commit_parameters()

    return radiative_transfer
```

Code example 7.4: Routine to initialize the `Mocassin` radiative-transfer code for an experiment with a single star as ionizing source in a homogeneous spherical cloud of molecular hydrogen gas.

```
def setup_abundancies(code):
    table = code.abundancies_table()
    for atom in table.keys():
        table[atom] = 0.0
    table["H"] = 1.0
    table["He"] = 0.1
    table["C"] = 2.2e-4
    table["N"] = 4.e-5
    table["O"] = 3.3e-4
    table["Ne"] = 5.e-5
    table["S"] = 9.e-6
```

After initializing the grid and the abundances, we set the rest of the initial conditions, such as the temperature of the ambient gas and the limits, to the frequency mesh,

```
radiative_transfer.parameters.initial_nebular_temperature = 6000.0 | units.K
radiative_transfer.parameters.high_limit_of_the_frequency_mesh = (
    15 | mocassin_rydberg_unit
)
radiative_transfer.parameters.low_limit_of_the_frequency_mesh = (
    1.001e-5 | mocassin_rydberg_unit
)
```

We then initialize the run-specific code parameters, such as the total number of photons, the convergence limit, and the number of ionization stages in the experiment,

```
radiative_transfer.parameters.total_number_of_photons = 10000000
radiative_transfer.parameters.total_number_of_points_in_frequency_mesh = 600
radiative_transfer.parameters.convergence_limit = 0.09
radiative_transfer.parameters.number_of_ionisation_stages = 6
```

Finally we initialize the grid in the radiative-transfer code and load the pre-evolved star into the code as the ionizing source,

```
radiative_transfer.grid.hydrogen_density = grid.hydrogen_density
radiative_transfer.particles.add_particle(star)
```

After all model- and code-specific parameters are set, the grid is initialized, and the ionizing source is defined, then we can start the event loop. The loop is controlled by setting a maximum value to the number of photons and tracking the degree to which convergence has been reached,

```
max_number_of_photons = (
    100 * radiative_transfer.parameters.total_number_of_photons
)
previous_percentage_converged = 0.0
```

The full event loop is shown in Code example 7.5. Figure 7.4 presents the radial temperature profile of the molecular cloud.

7.4 Validation

A number of standard tests can be used to validate the results of radiative-transfer codes in a steady state. Approximate analytic solutions exist for a few problems, and radiative-transfer codes are tested exhaustively against them. Every serious radiative-transfer code performs well on all of these tests.

These tests are important, and any code must pass them, but the real challenge lies in problems where the medium through which the radiation passes evolves dynamically in response to the radiation. Very few analytic (or even approximate analytic) solutions exist for such problems, so definitive tests in these cases are rare. There is, however, a large research community actively addressing these issues, and they have designed several standard tests to which every code should be expected to yield qualitatively similar solutions. Two excellent examples of large collaborative efforts in this area have been carried out by Iliev et al. (2006, 2009). These tests are limited to ionizing radiation passing through an optically thick medium, which is an important process and one of the hardest problems in radiation transfer, as discussed in Section 7.1.1.1. It is difficult, in this context, to compare methods that make different assumptions about the problem, but at least having a suite of tests allows a researcher to validate the results of a radiative-transfer code against earlier calculations.

```
    step = 0
    while percentage_converged < min_convergence:

        radiative_transfer.step()
        percentage_converged = radiative_transfer.get_percentage_converged()
        print(
            "percentage converged :",
            percentage_converged,
            ", step :",
            step,
            ", photons:",
            radiative_transfer.parameters.total_number_of_photons,
        )

        if previous_percentage_converged > 5 and percentage_converged < 95:
            convergence_increase = (
                percentage_converged - previous_percentage_converged
            ) / previous_percentage_converged
            if (
                convergence_increase < 0.2
                and radiative_transfer.parameters.total_number_of_photons
                < max_number_of_photons
            ):
                radiative_transfer.parameters.total_number_of_photons *= 2
        step += 1
        previous_percentage_converged = percentage_converged

```

Code example 7.5: Event loop for the radiative-transfer code.

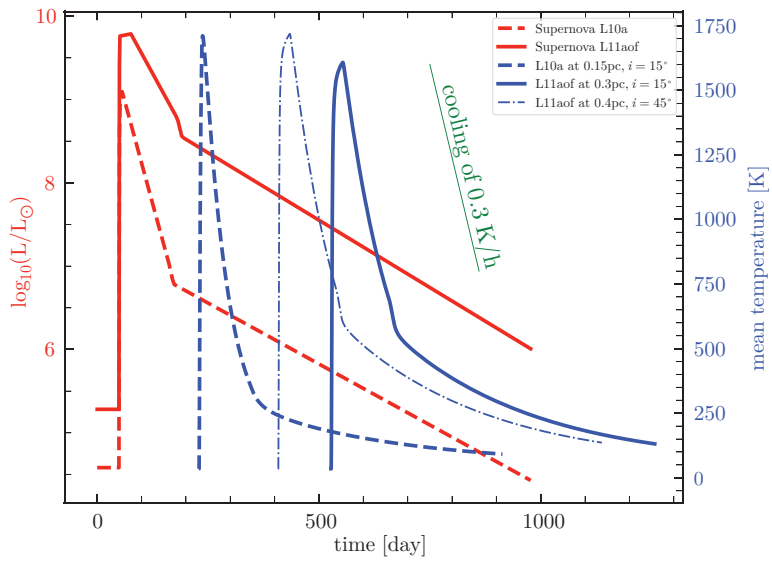

Figure 7.4. Radial electron temperature profile of a centrally illuminated H_2 cloud. The blue points indicate the initial temperature, the red points the final temperature. A script to generate this figure can be found at `amuse.examples.radiative_h2cloud`. It should take roughly 20 minutes to run.

Because testing codes against codes is the principal remaining validation strategy, we show here just one example. This is the last validation example in this book. Any further calculations using multiphysics solvers must be validated to the best degree possible, but we cannot offer any guarantee for the correctness of the results. The importance of radiative hydrodynamics across many theoretical and numerical astronomical problems has led to a wealth of papers on the cross-validation of methods and implementations. A few of the more recent ones include: Teyssier (2002); Leibrandt et al. (2005); Agertz et al. (2007); Iliev et al. (2009); Kolb et al. (2013); Ishii et al. (2017); Kim et al. (2017); Abe et al. (2018).

We run test suite number 5 of Iliev et al. (2009) and present the resulting temperature evolution in Figure 7.5. This test calculates the expansion of a HII region in a homogeneous medium using four different solvers (see also Pelupessy et al. (2013); for details). We simplify the problem here by adopting a solution with single-frequency radiative transfer to study the thermal evolution of the gas, but the same strategy should work with a full radiative solver. We carry out this test using `Fi` or `Gadget2` for the hydrodynamics solver and `SimpleX` or `SPHRay` for the radiative transfer. The coupling of the hydrodynamics code with the radiative-transfer solver is realized using the strategy discussed in Section 8.4.2.

Both radiative codes show different behavior in the fall-off of the temperature, mainly due to the different ways in which high-energy photons are treated. A similar divergence in the ionization-front profiles is observable in Iliev et al. (2009). The

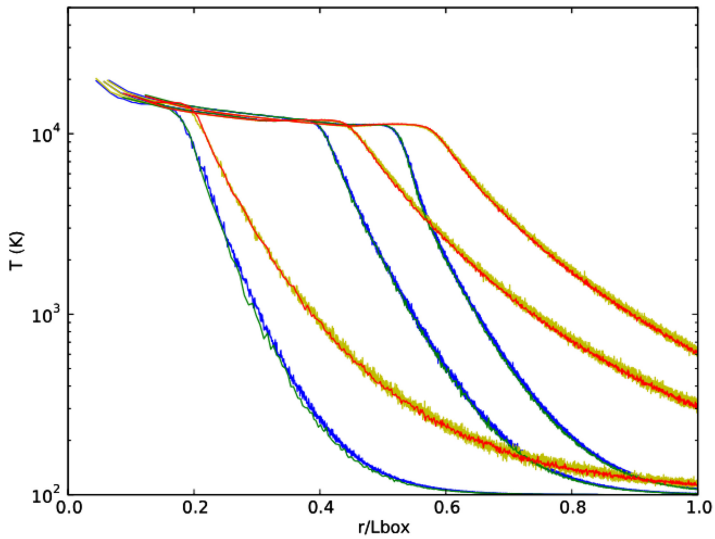

Figure 7.5. Expansion of an ionization front in an initially homogeneous medium (see test #5 of Iliev et al. 2009). We show here the temperature structure as a function of distance (normalized to the box size $L_{\rm box}$) after 10, 30, and 200 Myr. The source has a ionizing luminosity $L = 5 \times 10^{48}$ photos s^{-1}. Calculations are performed using `Fi` with `SimpleX` (blue curve), `Gadget2` with SimpleX (green), `Fi` with `SPHRay` (yellow), and `Gadget2` with `SPHRay` (red). A more extensive version of this test can be found in Pelupessy et al. (2013), see their Figure 22. Using script `amuse.examples.iliev_test5`, this figure can be generated in about 10 minutes on a fast workstation.

choice of hydrodynamic code has a minor effect on the temperature profile. Other differences can be traced to the specifics of the numerical treatments and the parameter settings of individual methods. The coupling with SPHRay, for example, results in more scatter in the temperature in Figure 7.5 than does the calculation with `SimpleX`. This is caused by using only 2×10^6 rays, which is a rather small number compared to the number of rays used in the Monte Carlo solver.

Unresolved results are generally hard to spot in radiative-transfer calculations, and can only be validated by repeating the simulation at higher resolution. The calculation presented in Figure 8.10 was performed using from 10^6 up to a maximum of 10^9 rays per hour. The latter calculation is expensive in terms of computer time, but the difference between the runs with 10^8 and 10^9 rays was smaller than the thickness of the line. One might then argue that 10^8 rays is sufficient. Note, however, that changing the resolution of the underlying SPH model would again change the results, and a new convergence test would be needed for validation. Therefore, the response of the results to the resolution depends not only on the resolution of the radiative-transfer calculation but also on the resolution of the underlying gas code. This poses interesting challenges for validating coupled radiative-hydrodynamics problems.

7.5 Assignments

7.5.1 Habitability of a Protoplanetary Disk

The habitability of a protoplanetary disk depends on the local temperature. We can calculate the disk's temperature structure using a radiative-transfer code acting on a gaseous disk. This assignment therefore consists of two parts: (1) obtaining a static hydrodynamical realization of a protoplanetary disk, and (2) allowing the disk to evolve. The radiative-transfer codes available in AMUSE, however, are dedicated to ionizing radiation, and calculating the temperature structure in the disk around a low-mass star will therefore not give the correct answer. In this example, we consider a disk around a high-mass star, which we can address using the available radiative-transfer codes.

1. Generate the initial conditions for a protoplanetary disk of 1% of the central star's mass, extending from 1 au to 100 au. Adopt a Toomre Q parameter of 2 (see Section 8.6.1), which will result in a relatively thin disk. In Section 6.4.1, we discussed how to generate such a disk model. Perform the simulation for a star of 10 $M_\odot$ at an age of 10 Myr (see Section 5.2.2). You should run a stellar evolution code to obtain the appropriate luminosity of the star.
2. Measure the temperature in the disk for a duration of 1 Myr, and make a plot of local temperature as a function of radius. Determine the extent of the so-called habitable zone in the disk—the region where the temperature lies between 273 K and 373 K (a broad range, but one in which Earth organisms are known to survive and thrive; see Miroshnichenko et al. 2001). Life also seems to be able to flourish in more extreme environments (Rothschild & Mancinelli 2001), and you may want to extend the range of the habitable zone to include psychrophilic organisms at temperatures below the freezing point of water (Cavicchioli et al. 2002).

3. For a more massive (brighter) star, the habitable zone moves outward. Plot the range of the habitable zone as a function of the luminosity of the central star. What can you say about the radial temperature structure within the habitable zone as the mass of the star increases, other than that it gets bigger and moves outward? Specifically, make a plot of the mean temperature in the habitable zone as a function of the distance to the star and as a function of the mass of the star.

7.5.2 Bumpy Disks

When running the disk with a bump problem in Section 6.4.1, it was clear that the bump shields part of the disk from the central irradiating source. As a consequence, the disk behind the bump remains cooler compared to its surroundings. One might wonder whether it is possible to create a habitable zone behind such a bump, or is the lifetime of the bump too short for life to develop? In order to prevent the bump from dispersing, we put a massive planet or a secondary star in the bump.

In this assignment, you will simulate a protoplanetary disk with a bump, in order to study the temperature evolution of the region behind the bump and compare this with the temperature elsewhere.

1. Generate the initial conditions for a protoplanetary disk with a bump around a 10 $M_\odot$ star. The mass of the disk should be 1% of the central star's mass, ranging from 1 au to 100 au. Adopt a Toomre Q parameter of 10, which will result in a relatively thick disk. Generate a Plummer density distribution with a mass of 10% of the disk and a radius of 10 au, and put it in a circular orbit at a distance of 50 au around the central star (see Section 6.4.1). While running, use `hop` (see Section 8.3.1) to keep track of the position, velocity, size, and mass of the bump.
2. Measure the temperature in the disk for as long as the bump has at least 30% of its initial mass. Plot the mean temperature in the bump as a function of time, and of the point diagonally across the disk at the other side and at the same distance from the central star. This will give you a sort of differential measurement of the temperature in the disk with and without bump.
3. Repeat the calculation for a 10 $M_\odot$ star at the terminal-age main sequence, at the beginning of the giant branch, and when it ascends the asymptotic giant branch.
4. Is there any moment in time at which the bump would be a better place to live than at the other side of the disk?

References

Abe, M., Suzuki, H., Hasegawa, K., et al. 2018, MNRAS, 476, 2664
Agertz, O., Moore, B., Stadel, J., et al. 2007, MNRAS, 380, 963
Altay, G., Croft, R. A. C., & Pelupessy, I. 2008, MNRAS, 386, 1931
Altay, G., Croft, R. A. C., & Pelupessy, I. 2011, SPHRAY: A Smoothed Particle Hydrodynamics Ray Tracer for Radiative Transfer, Astrophysics Source Code Library, ascl:1103.009
Baba, T. 2008, NaPho, 2, 465

Bohren, C. F., & Huffman, D. R. 1983, Absorption and Scattering of Light by Small Particles (New York: Wiley)
Cavicchioli, R., Siddiqui, K. S., Andrews, D., & Sowers, K. R. 2002, COBt, 13, 253
Chandrasekhar, S. 1960, Radiative Transfer (New York: Dover)
Delaunay, B. 1934, Bull. Acad. Sci. USSR(Vll), Classe Sci. Mat. Nat., 7, 793
Ercolano, B., Barlow, M. J., Storey, P. J., & Liu, X. W. 2003, MNRAS, 340, 1136
Fryxell, B., Olson, K., Ricker, P., et al. 2000, ApJS, 131, 273
Gerritsen, J. P. E., & Icke, V. 1997, A&A, 325, 972
Hernquist, L., & Katz, N. 1989, ApJS, 70, 419
Hubber, D. A., Ercolano, B., & Dale, J. 2016, MNRAS, 456, 756
Iliev, I. T., Ciardi, B., Alvarez, M. A., et al. 2006, MNRAS, 371, 1057
Iliev, I. T., Whalen, D., Mellema, G., et al. 2009, MNRAS, 400, 1283
Ishii, I. T., Ohnishi, D., Nagakura, G., Ito, G., Yamada, G., et al. 2017, JCoPh, 348, 612
Kim, J.-G., Kim, W.-T., Ostriker, E. C., & Skinner, M. A. 2017, ApJ, 851, 93
Kolb, S. M., Stute, M., Kley, W., & Mignone, A. 2013, A&A, 559, A80
Leibrandt, D. R., Drake, R. P., Reighard, A. B., & Glendinning, S. G. 2005, ApJ, 626, 616
Miroshnichenko, M. L., Hippe, H., Stackebrandt, E., et al. 2001, Extmo, 5, 85
Paardekooper, J. P., Kruip, C. J. H., & Icke, V. 2010, A&A, 515, A79
Pelupessy, F. I., van der Werf, P. P., & Icke, V. 2004, A&A, 422, 55
Pelupessy, F. I., van Elteren, A., de Vries, N., et al. 2013, A&A, 557, A84
Peraiah, A. 2001, An Introduction to Radiative Transfer: Methods and Applications in Astrophysics (Cambridge: Cambridge Univ. Press)
Plummer, H. C. 1911, MNRAS, 71, 460
Portegies Zwart, S. F., & Verbunt, F. 1996, A&A, 309, 179
Portegies Zwart, S. F., & Verbunt, F. 2012, SeBa: Stellar and Binary Evolution, Astrophysics Source Code Library, ascl:1201.003
Ritzerveld, J., & Icke, V. 2006, PhRvE, 74, 026704
Rothschild, L. J., & Mancinelli, R. L. 2001, Natur, 409, 1092
Strömgren, B. 1939, ApJ, 89, 526
Teyssier, R. 2002, A&A, 385, 337
Voronoi, G. 1908, JRAM, 134, 198